



CONDUCTOR: Dynamically orchestrating pipeline parallelism with multi-granularity control[☆]

Xingbo Dong^a, Ziyuan Liu^a, Yuezhe Yang^a, Yen-Lung Lai^b, Zhe Jin^a

^a Anhui Provincial International Joint Research Center for Advanced Technology in Medical Imaging, Anhui University, Hefei, 230093, Anhui, China

^b Department of Computing, Lee Kong Chian Faculty of Engineering and Science, Universiti Tunku Abdul Rahman, Sungai Long Campus, Kajang, Selangor, Malaysia

ARTICLE INFO

Keywords:

Pipeline parallelism
Load balancing
Performance optimization

ABSTRACT

Pipeline parallelism is a fundamental pillar of large scale model training yet its efficiency is frequently constrained by straggler induced pipeline bubbles. This issue is exacerbated by static scheduling approaches including handcrafted heuristics and Integer Linear Programming which are inherently brittle when facing real world execution time variance. In this work we introduce CONDUCTOR which is a dynamic and two tiered scheduling framework designed to virtually eliminate straggler induced bubbles under realistic stochastic conditions. The key technical insight is to decouple global and long horizon scheduling from local and instantaneous load balancing. At a coarse grain a reinforcement learning agent leverages millisecond scale inference to generate robust global schedules and adapts to runtime dynamics in scenarios where traditional static solvers are computationally intractable. At a fine grain we introduce a dynamic computation migration mechanism that resolves residual micro bubbles by offloading sub computations such as attention heads from transiently slower workers to faster ones within a single timestep. Evaluated on large scale language model training configurations our framework outperforms state of the art static scheduling baselines by 5% to 14% in throughput and demonstrates superior resilience to injected system noise and execution variance.

1. Introduction

Pipeline parallelism (PP) [1–4] has emerged as a fundamental pillar for training large-scale deep learning models, particularly Large Language Models (LLMs) with billions of parameters. By partitioning a model's layers across a distributed fleet of workers, PP enables computation to scale beyond the memory constraints of individual devices. However, the practical efficiency of this method is frequently bottlenecked by pipeline bubbles. These are idle periods where computational resources remain underutilized due to inter-stage data dependencies. A primary catalyst for these bubbles is the straggler problem [5], where execution time variances across different microbatches or hardware stages cause the entire pipeline to stall while waiting for the slowest task to complete. As illustrated in Fig. 1, even subtle fluctuations in computation time can propagate through the rigid dependency chain of the pipeline, leading to significant cumulative performance degradation.

Finding a global pipeline schedule that minimizes these stalls is a complex combinatorial problem, often proven to be NP-hard [6]. Existing static scheduling strategies generally fall into two categories [7], both of which struggle with the dynamic stochasticity of modern distributed systems. The first category comprises handcrafted heuristics such as the load-balancing schemes in GPipe [1] and the 1F1B strategy [2,3]. While these methods facilitate faster memory release, they rely on idealized assumptions of uniform computation time. In practice, factors such as the quadratic complexity of self-attention, the inherent asymmetry between forward and backward passes [8], and system-level noise such as network jitter or kernel variance consistently violate these assumptions. The second category involves optimization-based methods such as Integer Linear Programming (ILP) [9]. Although ILP can find theoretically optimal static plans, its exponential computational complexity makes it intractable for large-scale clusters. More importantly, static optima are brittle; they cannot adapt to the runtime perturbations that are inevitable in real-world GPU execution. While

[☆] This work was supported by the National Natural Science Foundation of China (Nos. 62306003, 62376003) and the Open Research Fund from Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), under Grant No. GML-KF-24-29.

* Corresponding author.

E-mail addresses: xingbo.dong@ahu.edu.cn (X. Dong), luckyzy98@gmail.com (Z. Liu), wa2214014@stu.ahu.edu.cn (Y. Yang), laiyl@utar.edu.my (Y.-L. Lai), jinzhe@ahu.edu.cn (Z. Jin).

<https://doi.org/10.1016/j.future.2026.108762>

Received 25 April 2026; Received in revised form 23 July 2026; Accepted 7 August 2026

Available online 20 August 2026

0167-739X/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

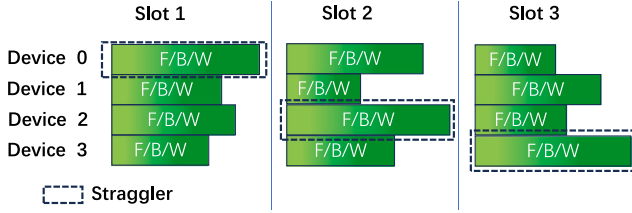


Fig. 1. In any given slot, the device with the longest execution time becomes the straggler for that step such as Device 2 in Slot 1. Because all devices must synchronize at the end of a slot before starting the next, this straggler dictates the pace of the entire pipeline. This constraint forces faster devices into idle states and creates performance bottlenecks.

recent advancements like DualPipe [10] and DynMo [11] optimize communication overlap or dynamic partitioning under their respective settings, robustly reducing residual bubbles under generalized stochastic conditions remains an open challenge.

To address these challenges, we propose a paradigm shift from static planning to a multi granularity dynamic control framework. Our key insight is that performance loss arises at different operational scales where macro level pipeline dependencies dictate the base bubble size and micro level execution variances exacerbate inefficiency at each step. Consequently, these factors must be decoupled and addressed with tools of corresponding granularity. We introduce CONDUCTOR, which is a two tiered framework designed for resilient and high throughput pipeline orchestration.

At a coarse grain, we employ a reinforcement learning (RL) agent to address macro-level scheduling. Unlike static ILP solvers that require extensive re-computation time, our pre-trained RL policy learns a generalized decision policy. It leverages millisecond-scale inference latency to provide near-instantaneous scheduling adjustments in response to runtime dynamics, such as network fluctuations or node preemptions. This ensures a level of robustness and real-time responsiveness that is unattainable by traditional static methods.

At a fine grain, recognizing that global policies cannot perfectly mitigate all localized hardware variances, we introduce a dynamic computation migration mechanism. Based on real-time profiling of preceding steps, this mechanism reactively offloads decomposable sub-computations such as multi-layer perceptron blocks or specific attention heads from transiently slower workers to faster ones. Migration is issued only when the modeled overlap condition is satisfied. Unmodeled communication, synchronization, and contention overheads may still leave residual stalls in a complete deployment.

The primary contributions of this work are summarized as follows:

1. **A Tractable RL-Based Pipeline Formulation:** We successfully formulate the dynamic pipeline scheduling problem as a Markov Decision Process (MDP). To manage the intractably large action space, we propose a two-phase training strategy: Behavioral Cloning based on a strong heuristic to bootstrap the agent, followed by Proximal Policy Optimization (PPO) refinement with Domain Randomization to ensure resilience against system noise.
2. **A Fine-Grained Computation Migration Mechanism:** We develop a variance-reduction strategy that utilizes a symmetric pairing algorithm to balance workloads within pipeline stages. We provide a formal analysis of the conditions under which the modeled migration communication can be masked by concurrent computation.
3. **Component-Level Implementation and Hybrid Evaluation:** We implemented the fine-grained migration module atop Megatron-LM and evaluated it using GPT-3 configurations of up to 14.6B parameters on 16 A100 GPUs, while the RL scheduler is evaluated in a hardware-profile-anchored offline simulator.

The measured Megatron-LM migration results yield throughput improvements of 5%–14% over the evaluated static baselines. We explicitly distinguish these measured module-level results from a monolithic RL-plus-migration Megatron-LM deployment, which has not yet been completed.

2. Quantitative analysis and modeling of performance variance

The efficiency of PP is fundamentally dictated by the temporal alignment of its constituent stages. As demonstrated in recent large-scale production traces [5], stragglers are a systemic issue rather than anomalous events, with approximately 42.5% of training jobs experiencing at least a 10% slowdown due to execution variance. In this section, we provide a quantitative taxonomy of the factors that transform deterministic model architectures into stochastic runtime systems.

2.1. Structural stage-level computational skew

Modern LLMs exhibit inherent workload imbalances that cannot be fully mitigated by uniform layer partitioning. In a typical PP setup, the terminal stages are heterogeneous by design. Specifically, the initial stage (PP Stage 0) uniquely handles token embedding, a memory-bandwidth-bound operation involving large table lookups, which lacks the compute-intensive self-attention kernels present in intermediate stages. Conversely, the final stage (PP Stage $N - 1$) is tasked with computing output logits and the cross-entropy loss, involving computationally expensive Softmax operations and high-dimensional matrix multiplications. While system designers attempt to balance the pipeline by redistributing Transformer layers, these non-divisible entry-exit tasks create a structural base bubble. Our profiling indicates that even with optimal layer partitioning, a residual imbalance of 3%–7% in execution time persists between the longest and shortest stages under static conditions.

2.2. Dynamic microbatch-level execution variance

A critical source of runtime variance stems from the architectural complexity of the Transformer block. Despite the use of sequence packing to maintain a constant total token count per microbatch, the execution time remains highly sensitive to the sequence length distribution within the batch [12]. This is primarily due to the self-attention mechanism, which possesses a quadratic complexity of $O(T^2d)$ relative to sequence length T and model dimension d [13]. For a sequence of length T , the ratio of floating-point operations (FLOPs) allocated to the Attention mechanism versus the Multi-Layer Perceptron (MLP) block can be formally expressed as:

$$\mathcal{R}_{Att} = \frac{4Td^2 + 2T^2d}{12Td^2 + 2T^2d} \quad (1)$$

$$\mathcal{R}_{MLP} = \frac{8Td^2}{12Td^2 + 2T^2d}. \quad (2)$$

Eq. (1) reveals that as T increases, the computational profile shifts non-linearly. In our empirical measurements using a hidden dimension of 4096, a single 8 K sequence required 3.616 ms, whereas eight 1 K sequences (totaling the same 8 K tokens) required only 0.569 ms. This 6.3× discrepancy underscores why static schedules, which assume microbatch homogeneity, fail to prevent transient bubbles during long-context training. Note that the decomposition in Eqs. (1)–(2) describes the overall compute composition of a Transformer block. In our later migration model for the weight-gradient pass, we only treat the MLP portion as migratable for implementation simplicity.

2.3. Fundamental computational asymmetry of F/B/W passes

The core computational phases of backpropagation, Forward (F), Backward-to-Input (B), and Backward-to-Weight (W), possess intrinsic FLOPs discrepancies. Theoretical and empirical analyses [2,8] confirm the execution time inequality: $T_B > T_F > T_W$. Specifically, the B-pass typically requires twice the FLOPs of the F-pass to compute gradients for the input activations. Static schedulers like 1F1B assume a fixed ratio between these passes; however, runtime factors such as operator fusion and cache locality cause these ratios to fluctuate. Any scheduling model that fails to account for this dynamic asymmetry will suffer from “dependency drift”, where small delays in the B-pass of a downstream stage cascade into large idle gaps for upstream stages.

2.4. Systemic stochasticity and communication jitter

Beyond deterministic computational factors, environmental factors in shared GPU clusters introduce significant noise. PP relies on frequent point-to-point transfers of activations and gradients, making it highly susceptible to network contention. The effective communication time, T_{comm} , is not a constant but a random variable affected by several factors [14]. Firstly, network jitter, manifested as bandwidth fluctuations in non-blocking Fat-Tree or Torus topologies due to traffic from concurrent jobs, contributes to unpredictable delays. Additionally, kernel performance variance arises from non-deterministic GPU kernel launch overheads and memory fragmentation, such as Python garbage collection pauses that can stall execution for dozens of milliseconds [5]. In summary, the compounding effect of structural skew, workload fluctuations, pass asymmetry, and systemic jitter renders static optimal schedules brittle. The traditional view—that all devices within a pipeline must synchronize or proceed in lock-step—effectively means the entire system is bottlenecked by the transient straggler of the most delayed stage. This quantitative gap necessitates a dynamic, multi-granularity control approach that can adapt to these variances in real-time.

3. Related work

The persistent challenge of pipeline bubbles and execution variance has driven the evolution of distributed training from rigid handcrafted heuristics to sophisticated adaptive orchestration frameworks. This section situates CONDUCTOR within the broader landscape of PP and contemporary adaptive systems as well as the application of RL for system level optimization.

3.1. Foundations of PP and the ubiquity of stragglers

PP has transitioned from synchronous foundations such as the GPipe framework [1] which established the paradigm of micro batching to increasingly fine grained task scheduling [12]. Subsequent advancements such as 1F1B and its interleaved variants optimized memory efficiency by early scheduling of backward passes and became the de facto standard in major frameworks [3,15]. Further explorations in distributed computing have introduced adaptive partitioning systems such as iPipe [16] to maximize throughput and addressed the trade-off between staleness and synchronization overhead in time efficient training architectures [17]. To further minimize device idleness the Zero Bubble methods proposed splitting the backward pass into activation gradient computation and weight gradient computation to decouple the critical path and allow for flexible bubble filling [8].

In real world production clusters these static assumptions are frequently violated by the straggler phenomenon. A longitudinal study conducted on large scale training traces indicates that stragglers are a systemic bottleneck rather than anomalous events where approximately 42.5% of training jobs experience at least a 10% slowdown due to execution variance [5]. These delays often arise from factors such

Table 1

Qualitative comparison of pipeline parallelism methods under their reported execution assumptions. The table is intended for technical positioning rather than as a substitute for aligned throughput experiments.

Method	Sched. type	Straggler aware	Training Req.	Memory overhead	Adaptive runtime
GPipe	Static	No	No	High	No
1F1B	Static	No	No	Medium	No
1F1B-I	Static (IL)	No	No	Medium	No
Zero Bubble	Static	No	No	Medium	No
DualPipe	Static (Bi)	Partial	No	High	No
iPipe	Adaptive partition	Partial	No	Medium	Limited
DynMo	Semi-dyn.	Partial	No	Medium	Limited
Adaptra	Hybrid adaptive	Yes	No	Medium	Yes
CONDUCTOR	Full Dyn.	Yes	Yes (~2m)	Same as ZB	Yes

as GPU memory fragmentation and sequence length imbalances in long context training. Static schedules including theoretically optimal ones derived via ILP lack the reactive capacity to adapt to these stochastic perturbations which often leads to cascading idle periods that significantly diminish throughput.

3.2. Current adaptive systems and competitive baselines

Recent work has explored several approaches to mitigate workload imbalance in pipeline parallelism, but they operate under different control objectives and execution assumptions. DualPipe introduces a deterministic bidirectional schedule that injects micro-batches from both ends of a fixed pipeline to overlap communication and computation [10]. DynMo targets structurally dynamic LLM training and adapts the pipeline partition at a coarser reconfiguration timescale [11]. iPipe focuses on adaptive partitioning and execution-system reconfiguration for pipeline-parallel DNN training [16]. Adaptra directly targets communication stragglers by adapting the pipeline schedule and using CPU-side RDMA to avoid GPU head-of-line blocking [18].

CONDUCTOR addresses a different combination of control levels: RL-based global operation ordering and intra-slot migration of decomposable computation in response to transient runtime variation. The overlap with Adaptra is therefore closest at the level of straggler resilience, but the mitigation mechanism and execution path differ. An absolute throughput comparison would require aligned execution engines, model partitions, cluster topology, and straggler-injection protocols. We consequently use these methods for technical and applicability positioning and do not claim measured superiority over them in the present evaluation.

To position CONDUCTOR within this landscape, Table 1 compares representative pipeline parallelism methods across five dimensions: scheduling type, straggler awareness, training requirements, memory overhead, and runtime adaptability.

As shown in Table 1, the methods differ in scheduling direction, adaptation target, control granularity, and runtime mechanism. The present hardware comparison in Section 6 evaluates the migration module against strong static schedules. Full hardware comparisons against Adaptra, iPipe, DualPipe, and DynMo require aligned execution engines, model partitions, cluster assumptions, and perturbation protocols. We therefore retain the qualitative comparison and list aligned evaluation as future work rather than claiming a measured advantage over every adaptive system.

Enabling such responsiveness requires a scheduling mechanism capable of making low-latency decisions under uncertainty. This motivates the adoption of reinforcement learning, discussed in the next section.

3.3. RL in distributed system orchestration

The application of RL to scheduling problems has emerged as a robust alternative to traditional solvers. Recent studies have demonstrated the efficacy of learning driven algorithms in managing complex

multi tenant distributed environments where agents can achieve near optimal schemes under dynamic conditions where network latency and resource capacity fluctuate [18–20]. Such learning based approaches have also proven effective for microservice deployment in Kubernetes clusters by prioritizing latency reduction and avoiding distribution inequality [21]. Furthermore advancements in distributed scalable multi agent RL with dual exploration suggest that learning based policies can effectively navigate intractably large decision spaces [22].

CONDUCTOR contributes to this trajectory by successfully formulating the pipeline scheduling problem as a MDP and overcoming the action space complexity through heuristic guided exploration. By utilizing Behavioral Cloning on Zero Bubble heuristics followed by policy refinement with Domain Randomization our framework learns a strategy that is inherently resilient to the system noise that typically cripples static approaches [8,23]. Our work extends the utility of RL beyond macro scheduling by coupling it with a stall free computation migration mechanism. This synergy ensures that global policies handle large scale dependencies while micro level imbalances are resolved locally which establishes a new paradigm for resilient training in uncertain environments.

4. Dynamic pipeline scheduling with RL

Handcrafted heuristics such as Zero-Bubble-style schedules are effective under static assumptions, but they are less flexible when execution conditions deviate from the profiled case. Directly solving a new combinatorial scheduling problem online is also expensive. To bridge this gap, we adopt a heuristic-guided RL scheduler that preserves the structural prior of a strong baseline while learning to improve local scheduling decisions under dynamic conditions. Concretely, rather than learning the entire schedule from scratch, the agent is only invoked at decision points where multiple feasible operations are simultaneously available. This design substantially reduces the effective decision space and makes policy learning practical for pipeline execution.

4.1. Problem formulation as a MDP

We formulate pipeline scheduling as a MDP, denoted by $(S, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. At each decision step, the environment first advances simulation time to the next instant at which at least one operation becomes schedulable. The agent then selects one operation from the current feasible set, executes it, and repeats until all operations are scheduled.

State space. The state vector summarizes both the global execution context and the current candidate actions. To avoid overloading symbols, we use D for the number of pipeline devices and M for the number of micro-batches; the pass type is denoted by $q \in \{F, B, W\}$ in later migration equations. The state consists of four parts:

- **Device status** (3D dimensions): normalized device available time, current memory usage, and scheduled workload ratio for each device;
- **Micro-batch progress** (3M dimensions): binary completion indicators for the forward, backward, and weight-update stages of each micro-batch;
- **Global status** (4 dimensions): normalized current time, overall scheduling progress, estimated bubble ratio, and average memory utilization;
- **Candidate-event features**: for each candidate slot in the bounded action space, a 6-dimensional feature vector encoding operation type (3-dimensional one-hot), device index, micro-batch index, and normalized duration.

Since the maximum number of operation slots is $3DM$, the total state dimension is

$$\|S\| = 3D + 3M + 4 + 6 \times 3DM = 3D + 3M + 4 + 18DM. \quad (3)$$

This augmented representation allows the policy network to jointly reason about the current system status and the semantic meaning of each candidate action.

Action space. The action space is a bounded discrete space with an upper bound of $3DM$, corresponding to all possible operation slots in one schedule. At each decision step, however, only the currently feasible operations are valid. Therefore, we apply an action mask and construct the policy distribution only over the feasible subset. To keep the action semantics stable across states, feasible operations are sorted by device index, operation type priority, and micro-batch index before being mapped to action slots.

Transition dynamics and feasibility constraints. The environment transition is deterministic once an action is selected. The chosen operation is assigned a start time equal to the maximum of the current simulation time and the availability time of its assigned device, after which the device state and pipeline progress are updated accordingly. An operation is considered feasible only if all of the following conditions are satisfied: (i) its precedence constraints in the pipeline graph have been met, (ii) its assigned device is available at the current decision time, and (iii) the resulting memory usage does not exceed the device memory budget. Hence, memory is enforced as a hard feasibility constraint rather than a soft penalty term.

Reward design. We use a shaped reward consisting of an immediate decision reward and a terminal schedule-quality reward. The step reward is defined as

$$R_t^{\text{step}} = R_t^{\text{base}} + R_t^{\text{eff}} + R_t^{\text{zb}}, \quad (4)$$

where R_t^{base} is a small positive reward for making a feasible scheduling decision, R_t^{eff} encourages selecting shorter operations among the currently feasible candidates, and R_t^{zb} is a lightweight imitation bonus when the selected action agrees with the ZBPP heuristic. This shaping stabilizes early training and preserves a strong scheduling prior.

At the end of an episode, the scheduler receives a terminal reward that evaluates the quality of the complete schedule:

$$R^{\text{final}} = R^{\text{time}} + R^{\text{comp}} - R^{\text{bubble}} - R^{\text{balance}}, \quad (5)$$

where R^{time} rewards makespan reduction relative to the ZBPP baseline, R^{comp} encourages completing all operations, R^{bubble} penalizes low device utilization, and R^{balance} penalizes variance in device finish times. This reward structure directly aligns training with the optimization goals of reducing completion time, limiting pipeline bubbles, and mitigating device-level stragglers.

4.2. Policy learning with heuristic bootstrapping

To improve sample efficiency, we adopt a two-stage training strategy that combines heuristic supervision with policy optimization.

Stage 1: Behavioral cloning from ZBPP. We first collect expert demonstrations from the ZBPP scheduler. Importantly, the expert target is not an entire handcrafted sequence treated as an indivisible object; instead, at each decision point the agent observes the current feasible action set and learns to imitate the action selected by the ZBPP heuristic. This stage initializes the policy in a region of the parameter space that already encodes a strong scheduling prior.

Stage 2: PPO-based policy refinement. After behavioral cloning, the policy is refined using PPO. We use an actor-critic network with shared hidden layers, where the actor outputs a probability distribution over the masked feasible action subset and the critic estimates the state value. The PPO objective is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (6)$$

where $r_t(\theta)$ is the probability ratio between the updated and old policies, and $\hat{A}_t$ is the estimated advantage. The critic is optimized with a mean-squared error loss on discounted returns, and an entropy regularization term is added to maintain exploration.

Compared with learning a schedule from scratch, this two-stage scheme is substantially more stable: behavioral cloning quickly establishes a competent policy, while PPO further improves it by exploring decision alternatives that may outperform the heuristic baseline in the simulated environment.

Training environment. To ensure a fair comparison against the heuristic baseline, the RL environment is initialized from the same operation templates used by the ZBPP schedule, so both methods are evaluated on identical operation durations within each rollout. During training, the simulator can inject execution variability through its noise parameter, allowing the policy to observe a broader range of runtime conditions than a purely deterministic trace.

Reproducibility details. The policy is an ActorCritic multilayer perceptron. The shared trunk has two hidden layers with hidden dimension 128, and the actor and critic heads also use MLP hidden dimension 128. Behavioral cloning uses 64 expert rollouts for 10 epochs, with the ZBPP heuristic action used as the expert choice at each decision point. PPO refinement uses learning rate 3×10^{-4} , clipping parameter 0.2, discount factor 0.99, 10 PPO epochs, batch size 64, and the Adam optimizer. The loss combines the clipped PPO policy loss, a value-loss coefficient of 0.5, and an entropy coefficient of 0.01. No separate GAE hyperparameter or additional scalar reward coefficients are exposed in the inspected implementation records beyond the reward composition in Eqs. (4)–(5). The paper-aligned training budget is 500 episodes, 48,000 environment steps, and 10,000 policy updates per noise setting. Training uses this fixed budget rather than an early-stopping convergence criterion. The evaluated domain-randomization settings are runtime-noise levels of 0%, 10%, 20%, and 30%.

Code availability. The RL scheduler source code, paper-aligned configuration, and command-line entry point are available in an anonymized repository at <https://anonymous.4open.science/r/Conductor-B4FB/>.

4.3. Inference-time rescheduling

At inference time, the trained policy is used as a lightweight rescheduling module. Given the current pipeline state and the feasible operation set, the policy selects the next operation greedily over the masked action distribution. Because the agent only operates at decision points and does not re-solve the full combinatorial problem from scratch, the overhead of schedule generation remains low.

This design is particularly suitable for perturbed execution conditions, where a static schedule may no longer be ideal. In such cases, the scheduler can use the updated runtime state to continue generating the remaining execution order online. Fig. 2 illustrates two representative cases. Fig. 2(a) shows that the learned policy can reduce idle bubbles caused by asymmetric forward/backward execution times, while Fig. 2(b) shows the benefit of adaptive rescheduling under bursty communication perturbations.

5. Dynamic intra-slot computation balancing

While the global orchestration policy minimizes the macro level makespan significant residual inefficiency persists due to execution time variance within parallel computation slots. The completion time of any given slot t for a specific computational pass P where P belongs to the set of forward and backward and weight update operations is bottlenecked by the slowest worker. This straggler effect is defined as $T_{col,t}^P = \max_i T_{i,t}^P$ and our primary objective is to minimize this value by dynamically redistributing the intra-slot workload.

Algorithm 1 Heuristic-guided PPO training for dynamic pipeline scheduling

Input: Initial policy parameters θ , value parameters ϕ , ZBPP scheduler, simulator environment

- 1: **Stage 1: Behavioral Cloning**
- 2: **for** rollout $i = 1$ to N_{BC} **do**
- 3: Reset the scheduling environment;
- 4: **while** the episode is not terminated **do**
- 5: Advance simulation time to the next decision point;
- 6: Enumerate the current feasible action set $\mathcal{A}_t^{\text{feasible}}$;
- 7: Query the ZBPP heuristic for the expert choice $a_t^* \in \mathcal{A}_t^{\text{feasible}}$;
- 8: Store the pair (s_t, a_t^*) ;
- 9: Execute a_t^* and update the environment;
- 10: **end while**
- 11: **end for**
- 12: Optimize π_θ by minimizing the cross-entropy loss on the collected expert pairs;
- 13: **Stage 2: PPO Refinement**
- 14: **for** episode $k = 1$ to K **do**
- 15: Reset the scheduling environment;
- 16: **while** the episode is not terminated **do**
- 17: Advance simulation time to the next decision point;
- 18: Enumerate feasible actions $\mathcal{A}_t^{\text{feasible}}$ and apply action masking;
- 19: Sample action $a_t \sim \pi_\theta(\cdot | s_t)$ over the feasible subset;
- 20: Execute a_t , receive reward R_t , and store (s_t, a_t, R_t, s_{t+1}) ;
- 21: **end while**
- 22: Compute discounted returns and advantage estimates using V_ϕ ;
- 23: Update the actor by optimizing Eq. (6);
- 24: Update the critic by minimizing the value regression loss;
- 25: **end for**

5.1. Decomposable computation and pass specific modeling

Achieving precise load balancing requires a decomposable model of transformer computation that accounts for the distinct behavior of different layers. We partition each primary pass P into its core constituents including self attention and the multi layer perceptron block. The total execution time for device i in slot t for pass P is formally modeled as the summation of these components and the communication latency

$$T_{i,t}^P = (T_{i,t}^P)^{Att} + (T_{i,t}^P)^{MLP} + T_{i,t}^{comm} \quad (7)$$

where component execution times are derived from the live profiling phase to capture real time hardware fluctuations. A key modeling assumption for the weight gradient pass is that its computation is primarily concentrated in the multi layer perceptron component because the self attention mechanism possesses minimal direct parameter gradients which allows us to assume $(T_{i,t}^W)^{Att} \approx 0$ consistent with prior efficiency studies [24].

5.2. Variance reduction via symmetric pairing

To minimize the maximum execution time across the pipeline we employ a variance reduction strategy through a symmetric pairing algorithm. The adoption of slot level synchronization ensures that training semantics remain consistent with standard synchronous optimization which simplifies the convergence analysis while allowing CONDUCTOR to focus on mitigating the hardware induced stragglers that would otherwise stall the entire pipeline hypercube. For a given pass P the compute devices are sorted based on their execution times to yield an ordered permutation π such that $T_{\pi(1),t}^P \leq \dots \leq T_{\pi(N),t}^P$. We then form $\lfloor N/2 \rfloor$ pairs by matching the k th fastest device with the k th slowest worker. For each resulting pair $k = (\pi(k), \pi(N-k+1))$ the target transfer

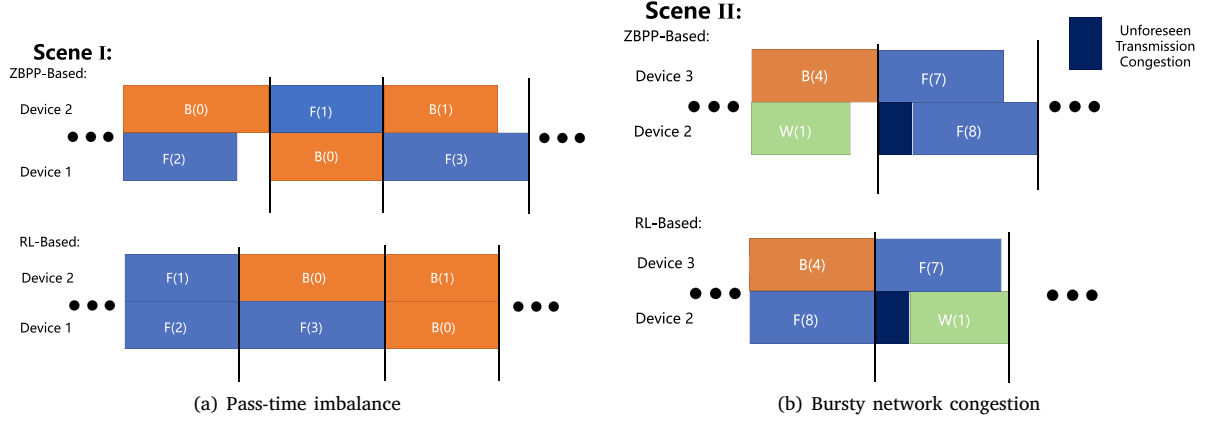


Fig. 2. Comparison between the ZBPP and the RL-based rescheduling policy under representative runtime perturbations.

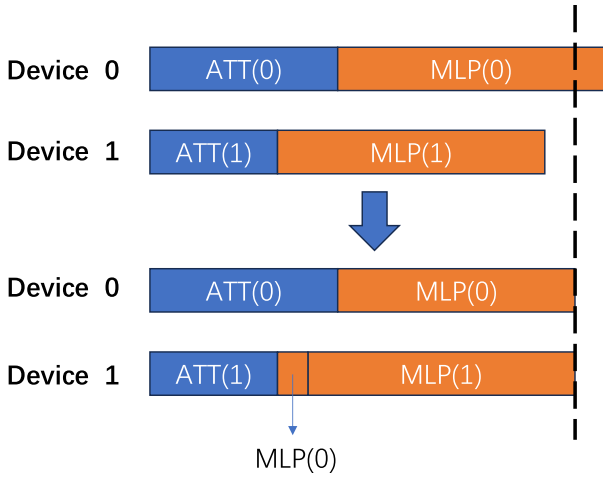


Fig. 3. Slower devices offload part of their computation to faster devices.

load $T_{transfer}^{(k),P}$ represents the amount of computation required to equalize their execution times defined as

$$T_{transfer}^{(k),P} = \frac{T_{\pi(N-k+1),t}^P - T_{\pi(k),t}^P}{2} \quad (8)$$

This calculated load serves as the deterministic target for the subsequent migration process as illustrated in Fig. 3.

5.3. Hybrid migration strategy and overlap guarantees

The target load is realized by intelligently partitioning the underlying computations of the slower device in each pair and the strategy is tailored to the specific characteristics of each pass. For forward and backward passes we utilize a hybrid prioritized strategy that first attempts to satisfy the target load by migrating discrete attention heads which are computationally self contained. The number of integer heads to offload $h^{(k),P}$ is determined as

$$h^{(k),P} = \lfloor \frac{T_{transfer}^{(k),P}}{T_{head}^{(k),P}} \rfloor \quad (9)$$

which represents the maximum number of heads that can be migrated without exceeding the target. Any residual transfer load $T_{rem}^{(k),P}$ remaining after head migration is then addressed by offloading a continuous fraction of the multi layer perceptron computation where the splitting

Algorithm 2 The Pseudo-code for the dynamic intra-slot computation migration

Input : Set of devices $\mathcal{D}$, Target pass P , Profiled times $T_{i,t}^P$, Overlap bound B

- 1: Sort devices in $\mathcal{D}$ based on $T_{i,t}^P$ to obtain permutation π ;
- 2: **for** $k = 1$ to $\lfloor |\mathcal{D}|/2 \rfloor$ **do**
- 3: Identify $device_{fast} \leftarrow \pi(k)$ and $device_{slow} \leftarrow \pi(N - k + 1)$;
- 4: Compute $T_{transfer}^{(k),P}$ using Eq. (8);
- 5: **if** P is Forward or Backward Pass **then**
- 6: Calculate attention head count $h^{(k),P}$ via Eq. (9);
- 7: Compute residual load $T_{rem}^{(k),P} \leftarrow T_{transfer}^{(k),P} - h \cdot T_{head}$;
- 8: Determine splitting ratio $\alpha^{(k),P}$ via Eq. (10);
- 9: **else** ▷ Weight Update Pass
- 10: $h^{(k),P} \leftarrow 0$;
- 11: $\alpha^{(k),P} \leftarrow \min(1, T_{transfer}^{(k),P} / T_{slow}^{MLP})$;
- 12: **end if**
- 13: **if** $\text{CommunicationCost}(h, \alpha) \leq B$ **then**
- 14: Offload h heads and α fraction of MLP to $device_{fast}$;
- 15: Execute migrated sub-computation concurrently;
- 16: **end if**
- 17: **end for**

ratio $\alpha^{(k),P}$ is calculated as

$$\alpha^{(k),P} = \min(1, \frac{T_{rem}^{(k),P}}{(T_{\pi(N-k+1),t}^P)^{MLP}}) \quad (10)$$

In contrast the weight update pass migration simplifies to only splitting the multi layer perceptron computation because of the absence of self attention load. Crucially this re-balancing is performed only if the induced communication overhead meets the stall-free overlap bound derived in Appendix A which ensures that the transfer time is fully masked by ongoing local computation. The complete execution logic for this fine grained balancing is detailed in Algorithm 2.

5.4. Analysis of stall-free computation migration

This section provides a concise formal analysis of the conditions for stall-free migration. Forward, backward-input, and weight-gradient migration follow the same dependency-preserving scheduling construction: a decomposable operator is assigned to a paired worker, its inputs are transferred, remote computation is overlapped with the source worker's remaining local computation, and the required result is returned before the dependent operation. We derive the bound for the forward MLP as a representative case because it exposes this shared

communication/computation trade-off without repeating the same construction for all three passes. The same notation is then instantiated for backward-input and weight-gradient migration by replacing the forward compute term, migrated fraction, and transfer volume with the corresponding pass-specific profiled quantities.

5.4.1. The stall-free overlap bound

For the master rank to avoid stalling, the critical path of the remote task must complete no later than the master's concurrent local computation. Let B_{eff} be the effective token count in the migrated micro-batch, s be the number of bytes per element, d be the hidden dimension, d_{ffn} be the MLP intermediate dimension, B_{net} be the effective bidirectional network bandwidth, and F_{eff} be the measured effective accelerator throughput for the relevant pass. This critical path includes sending activations and weights (C_{send}), remote execution ($T_{\text{remote-fwd}}$), and receiving the result (C_{recv}). The stall-free condition is $C_{\text{send}}(\alpha) + T_{\text{remote-fwd}}(\alpha) + C_{\text{recv}} \leq T_{\text{local-fwd}}(\alpha)$.

Assuming linear scaling with the fraction α , where $T_{\text{slow-fwd}}$ is the time for the full MLP forward pass on one GPU, this inequality simplifies to the following upper bound for the migratable fraction,

$$\alpha \leq \frac{T_{\text{slow-fwd}} - (C_{\text{send,act}} + C_{\text{recv}})}{2T_{\text{slow-fwd}} + C_{\text{send,weights}}} = \frac{\frac{4B_{\text{eff}}dd_{ffn}}{F_{\text{eff}}} - \frac{2B_{\text{eff}}ds}{B_{\text{net}}}}{\frac{8B_{\text{eff}}dd_{ffn}}{F_{\text{eff}}} + \frac{2dd_{ffn}s}{B_{\text{net}}}} \quad (11)$$

This final equation reveals that the maximum stall-free migration fraction is fundamentally determined by the ratio of effective computational throughput (F_{eff}) to network bandwidth (B_{net}), scaled by the geometric properties of the model ($B_{\text{eff}}, d, d_{ffn}$) and the data type size (s).

5.4.2. Case study: Two-layer MLP on an A100 cluster

We quantify α_{max} for our target configuration ($d_{\text{model}} = 5120, d_{ffn} = 20480, B_{\text{eff}} = 4096$) on a 100 GB/s network using BF16 precision. The commonly quoted 312 TFLOP/s value corresponds to peak A100 Tensor Core throughput rather than guaranteed sustained application throughput. We therefore use it only as a conservative illustrative case for the stall-free overlap test: peak throughput gives the shortest compute window, and any lower sustained throughput would increase the local compute time available to hide communication.

- **Total Forward Compute Time** ($T_{\text{slow-fwd}}$): The two-layer MLP forward pass requires 1.718×10^{12} FLOPs, not 10^{15} FLOPs. Under the peak-throughput illustrative case of 312 TFLOP/s, this corresponds to approximately 5.51 ms. This is the shortest compute window considered in the case study; a lower sustained throughput would make $T_{\text{slow-fwd}}$ larger.
- **Communication Times:**
 - Activation Send ($C_{\text{send,act}}$): Transferring the 41.9 MB input activation takes ≈ 0.42 ms.
 - Result Receive (C_{recv}): Receiving the 41.9 MB output result also takes ≈ 0.42 ms.
 - Full Weights Send ($C_{\text{send,weights}}$): Transferring the full $2 \times 209.7 = 419.4$ MB remote weight slices (for $\alpha = 1$) would take ≈ 4.19 ms.

Substituting the peak-throughput illustrative values into Eq. (11):

$$\alpha_{\text{max}} \approx \frac{5.51 - (0.42 + 0.42)}{2 \times 5.51 + 4.19} = \frac{4.67}{15.21} \approx 0.307$$

Conclusion: The illustrative calculation shows that up to 30.7% of the MLP's computation can be offloaded to a remote worker even under the peak-throughput assumption. Because this assumption minimizes $T_{\text{slow-fwd}}$, it gives a conservative communication-overlap screen. If the measured sustained throughput is lower than the peak value,

the local compute window becomes longer and the feasible stall-free migration fraction would not decrease under the same communication assumptions. For backward-input and weight-gradient migration, the shared dependency-preserving construction is applied with pass type $q \in \{B, W\}$: the forward quantities in the overlap inequality are replaced by the profiled pass-specific compute time T_{slow}^q , migrated fraction α_q , and pass-specific activation, weight, or gradient transfer volume. The forward MLP is therefore the representative derivation of the common mechanism; each backward migration remains admissible only when its own profiled communication and dependency constraints satisfy the corresponding inequality.

5.5. Theoretical analysis of pipeline efficiency and bubble rate

To rigorously quantify the efficacy of CONDUCTOR, we provide a formal mathematical analysis of the pipeline bubble rate under our multi-granularity control framework. The inherent efficiency of a pipeline is inversely proportional to the proportion of idle time (bubbles) within the total execution makespan.

Let D denote the set of D devices (pipeline stages) and N denote the total number of microbatches. The theoretical ideal compute time for the entire training step, assuming zero communication overhead and perfect load balancing, is defined as $T_{\text{ideal}} = \sum_{t=1}^T \sum_{i \in D} \mathbb{E}[T_{i,t}]$, where T represents the total number of scheduling slots (time steps) to complete the forward, backward, and weight update passes.

5.5.1. Bubble rate in static synchronous pipelines

In traditional synchronous schedules (e.g., 1F1B or Zero Bubble), the execution advances in slots. Due to strict intra-slot synchronization, the time cost of any slot t for a specific pass P , denoted as $T_{\text{col},t}^P$, is completely bottlenecked by the straggler device. Based on the ordered permutation π defined in Section 5.2, the slot execution time is constrained by the slowest device:

$$T_{\text{col},t}^P = \max_{i \in D} T_{i,t}^P = T_{\pi(D),t}^P \quad (12)$$

The total makespan of a static pipeline under realistic stochastic conditions, denoted as M_{static} , is the accumulation of these maximum slot times plus the macro-level structural bubbles $B_{\text{macro}}^{\text{static}}$ caused by pipeline warm-up, cool-down, and forward-backward pass asymmetry. The static pipeline bubble rate $R_{\text{bubble}}^{\text{static}}$ is formulated as:

$$R_{\text{bubble}}^{\text{static}} = 1 - \frac{T_{\text{ideal}}}{D \cdot M_{\text{static}}} = 1 - \frac{T_{\text{ideal}}}{D \cdot \left(\sum_{t=1}^T T_{\pi(D),t}^P + B_{\text{macro}}^{\text{static}} \right)} \quad (13)$$

Eq. (13) reveals that any variance across $T_{i,t}^P$ forces faster devices into idle states, where the idle penalty for device i at step t is exactly $(T_{\pi(D),t}^P - T_{i,t}^P)$.

5.5.2. Macro-level bubble reduction via RL agent

At the macro level, the RL agent policy $\pi_{\theta}(s_t)$ is trained to reduce B_{macro} by optimizing the execution sequence. By navigating the MDP formulated in our methodology, the agent can generate a dynamic task sequence that breaks the rigid symmetric assumptions of 1F1B. Let $B_{\text{macro}}^{\text{RL}}$ be the structural bubble time under the RL policy. Because reward shaping alone does not mathematically guarantee improvement for every instance, we define the realized macro-level reduction as an empirical quantity:

$$B_{\text{macro}}^{\text{RL}} = B_{\text{macro}}^{\text{static}} - \Delta B_{\text{macro}}, \quad \Delta B_{\text{macro}} = B_{\text{macro}}^{\text{static}} - B_{\text{macro}}^{\text{RL}} \quad (14)$$

where ΔB_{macro} is positive only when the learned policy improves the schedule on a given instance. In the experiments, this quantity is reported empirically rather than assumed by theory.

5.5.3. Micro-level variance reduction and formal overlap bound

While RL optimizes ΔB_{macro} , the dynamic computation migration minimizes the intra-slot straggler penalty. For the k th pair $(f, s) = (\pi(k), \pi(D - k + 1))$, where f is the faster device and s is the straggler, CONDUCTOR offloads a target load $T_{transfer}^{(k),P}$.

Assuming the formal stall-free overlap bound is met (i.e., communication cost $T_{comm} \leq T_{transfer}^{(k),P}$), the communication overhead is fully masked by the ongoing local computation on device f . The post-migration execution time for the straggler device s and the faster device f becomes:

$$\hat{T}_{s,t}^P = T_{s,t}^P - T_{transfer}^{(k),P} \quad (15)$$

$$\begin{aligned} \hat{T}_{f,t}^P &= \max \left(T_{f,t}^P + T_{transfer}^{(k),P}, T_{f,t}^P + T_{comm} \right) \\ &= T_{f,t}^P + T_{transfer}^{(k),P} \end{aligned} \quad (16)$$

By equating the target loads, we arrive at the balanced execution time for both devices:

$$\hat{T}_{s,t}^P = \hat{T}_{f,t}^P = \frac{T_{\pi(D-k+1),t}^P + T_{\pi(k),t}^P}{2} \quad (17)$$

Crucially, the new bottleneck time for slot t under CONDUCTOR, denoted as $\hat{T}_{col,t}^P$, is strictly less than the original static bottleneck, provided that variance exists:

$$\hat{T}_{col,t}^P = \max_k \left(\frac{T_{\pi(D-k+1),t}^P + T_{\pi(k),t}^P}{2} \right) \leq T_{col,t}^P - \Delta_{var,t} \quad (18)$$

where $\Delta_{var,t} = T_{\pi(D),t}^P - \hat{T}_{col,t}^P$ is the direct reduction in execution variance for step t .

5.5.4. Comprehensive pipeline efficiency formulation

Synthesizing the macro and micro-level optimizations, the total makespan of CONDUCTOR, $M_{Conductor}$, can be written conditionally as:

$$M_{Conductor} = M_{static} - \Delta B_{macro} - \sum_{t=1}^T \Delta_{var,t} + \Omega_{unmasked}, \quad (19)$$

where $\Omega_{unmasked} \geq 0$ captures any communication, synchronization, or runtime overhead that is not fully masked by concurrent computation. The idealized expression in which $\Omega_{unmasked} = 0$ is used only when the overlap bound is satisfied.

Consequently, using Eq. (19), the final bubble rate is:

$$R_{bubble}^{Conductor} = 1 - \frac{T_{ideal}}{D \cdot M_{Conductor}}. \quad (20)$$

Proposition 1 (Conditional Efficiency Bound). *If the learned scheduler produces a nonnegative empirical macro-bubble reduction ($\Delta B_{macro} \geq 0$), if every migrated sub-computation satisfies the pass-specific stall-free overlap bound so that $\Omega_{unmasked} = 0$, and if the requested load is decomposable enough to realize $\Delta_{var,t} > 0$, then the resulting makespan is no larger than the corresponding static makespan for the same profiled operation times.*

Eq. (20) explains how the empirical throughput gains in Section 6 can arise when the two reduction terms dominate any unmasked overhead. The structural term ΔB_{macro} addresses architectural asymmetry, while the variance term $\sum \Delta_{var,t}$ mitigates systemic stochasticity and network jitter. We no longer present this analysis as a convergence proof; instead, it states the formal conditions under which the proposed mechanisms reduce the static-pipeline makespan.

This proposition characterizes a selected schedule and does not guarantee that PPO will produce an improving candidate on every instance. The empirical benefit is expected to be scenario dependent: when the original schedule already has little runtime variation or few bubbles, the gain may approach zero, whereas more severe perturbations create more room for adaptive scheduling. A practical deployment can compare the estimated makespan of the learned candidate with the static ZBPP candidate and retain the static schedule when the learned candidate is not better. This baseline-fallback rule is an engineering safeguard suggested by the analysis; it is not assumed to have been implemented or evaluated in the current prototype.

6. Experiments

6.1. Setup

We adopt a hybrid evaluation methodology appropriate to an academic prototype. First, to validate the fine-grained computation balancing mechanism, we implement it directly on top of Megatron-LM [15] and measure module-level training throughput on GPT-3 style models [25] using up to 16 NVIDIA A100 GPUs. Second, small clusters and preprocessed training inputs are typically comparatively stable and do not permit controlled, repeated reproduction of the wide range of irregular workloads, abrupt delays, and network bursts expected during long-running training on much larger clusters. We therefore evaluate the RL global scheduler in an offline simulator whose base operation and communication parameters are anchored to the available Megatron-LM hardware profiles. The simulator is used to study controlled runtime variation and larger simulated device counts; it complements rather than replaces hardware validation and does not constitute a fully integrated RL-plus-migration Megatron-LM runtime.

For the simulator-based experiments, we use 4 devices, 8 micro-batches, a micro-batch size of 32, and a maximum memory budget of 4.0. The RL agent is trained with PPO for 500 episodes under four noise settings, namely 0%, 10%, 20%, and 30%. The migration communication-latency multiplier is set to 2.0. While the RL agent is evaluated within an event-driven simulator, we initialize the simulator from hardware profiles extracted from our Megatron-LM runs and report the remaining validation boundary explicitly below.

Simulator parameter anchoring and validation boundary. The simulator uses the same default setting as the paper experiments: 4 devices, 8 micro-batches, micro-batch size 32, maximum memory budget 4.0, and a migration communication-latency multiplier of 2.0. Its base operation times and communication parameters are initialized from the available Megatron-LM hardware profiles. However, the supplementary experiment records available for this revision do not include an independent held-out Megatron-LM trace split collected under the same profiling protocol. We therefore do not report MAPE, RMSE, or rank-correlation calibration scores in this revision, because doing so would require estimating validation errors from non-held-out or non-matched traces. Instead, we state this evidence boundary explicitly and report the available simulator stress tests below; held-out trace calibration remains an important future validation step.

Runtime-noise model. The linear-noise experiments use noise levels of 0%, 10%, 20%, and 30%. Compute-side perturbations represent effects such as kernel-runtime fluctuation, heterogeneous device speed, and irregular per-operation workload. In the supplementary stress runner, heavy-tailed compute delays are generated independently for each simulated operation using a Pareto perturbation with shape 2.5 and scale 0.08 after centering the sampled tail term; the multiplier is clipped below at 0.25. Communication-side perturbations represent transient network delay and concurrent offload contention. Network microbursts are injected independently on non-weight-update operations by multiplying the communication component by 4.0 with probability 0.15. The heterogeneous-capacity stress uses fixed device-capacity multipliers {1.00, 0.82, 1.18, 0.70, 1.08, 0.90, 1.25, 0.76} and independent uniform jitter in $[-0.10, 0.10]$ for each operation. Thus, the current stress models do not encode temporal correlation across operations or correlated failures across devices, and their parameters are not fitted to a production trace. They are controlled sensitivity tests for classes of compute delay, communication bursts, and device heterogeneity rather than a generative model of a particular production cluster.

Table 2

Repeated simulator trials at 4 devices, 8 micro-batches, and 10% noise.

Algorithm	n	Completion time (ms)	Bubble ratio
1F1B	5	433.5113 $\pm$ 3.7613	41.10% $\pm$ 0.37 pp
ZB-2p	5	345.2625 $\pm$ 3.6210	26.04% $\pm$ 0.47 pp
ZB-2p+Migration	5	258.6168 $\pm$ 1.4827	1.27% $\pm$ 0.46 pp

Repeated trials and statistical reporting. We repeat one representative simulator slice — the 4-device/8-microbatch/10% noise setting — across 5 random seeds and report mean $\pm$ 95% confidence intervals using $1.96\sigma/\sqrt{n}$. Table 2 shows that the migration approximation remains close to the paper’s default 10% noise result: ZB-2p+Migration obtains 258.6168 $\pm$ 1.4827 ms completion time and 1.27% $\pm$ 0.46 percentage points residual bubble ratio. This repeated-trial table does not provide uncertainty estimates for every RL configuration or for the Megatron-LM hardware measurements; those results remain point estimates and are interpreted accordingly.

Scalability stress. To examine simulator behavior beyond the default 4-device setting, we additionally run scale sweeps up to 64 simulated devices. The 8-, 16-, and 32-device results use $n = 3$ seeds, while the 64-device setting is a single-seed smoke test because scheduler runtime is substantially higher at that scale. Table 3 is therefore a simulator stress test, not evidence that CONDUCTOR has achieved hardware scalability on a 64-device or industrial-scale cluster. The current hardware evidence remains limited to the Megatron-LM migration-module experiments on up to 16 A100 GPUs.

We compare CONDUCTOR with several representative pipeline schedules. ZB-1p [8] constrains activation memory to pMB , where p is the number of pipeline stages and MB denotes the memory of one micro-batch; it theoretically matches the peak memory of 1F1B. ZB-2p constrains activation memory to $2pMB$, which is the smallest memory budget that empirically approaches zero bubble. We also include the standard 1F1B schedule [4] and its interleaved variant 1F1B-I [15]. For the simulator-based analysis, we further consider ZB-2p+RL, which enables only RL-based scheduling, and ZB-2p+Migration, which enables only computation migration. Our full simulator method enables both components together. As detailed in Section 3.2, DualPipe uses a deterministic bidirectional schedule, DynMo and iPipe adapt partitions at different control timescales, and Adaptra uses schedule adaptation and CPU-side RDMA for communication stragglers. We do not report their absolute hardware throughput without aligned execution engines, model partitions, cluster topology, and perturbation protocols, and we do not claim measured superiority over them.

6.2. Effectiveness of RL scheduling

Fig. 4 compares the pure ZBPP ordering and the RL-driven dynamic scheduling result. The learned scheduler introduces little additional idle time while improving the execution order, indicating that RL can construct an effective schedule for reducing pipeline stalls.

6.3. Megatron-LM performance of computation balancing

We evaluate the computation-balancing mechanism by comparing common pipeline-parallel schedules with and without migration while fixing model size, batch size, and sequence length. This hardware experiment validates the fine-grained migration module, not a fully integrated RL-plus-migration Megatron-LM runtime. Table 4 reports throughput and memory usage.

Across all three model sizes, the computation-balancing module consistently improves throughput while preserving the same memory footprint as the corresponding baseline schedule. Compared with ZB-2p, migration improves throughput from 14.5 to 15.1 samples/GPU/s on the 1.5B model, from 4.32 to 4.51 on the 6.2B model, and from 1.81

to 1.90 on the 14.6B model. These gains come from reducing intra-slot runtime imbalance: the migration module redistributes decomposable work within the same pipeline slot, thereby shortening the tail stage and reducing bubble time. A full end-to-end Megatron-LM run of the combined RL scheduler and migration module is beyond the current evidence scope and remains a future integration target.

6.4. RL training cost and scheduling overhead

To quantify the cost of training the scheduler, we fix the training budget to 500 episodes for all noise settings, corresponding to 48,000 environment steps and 10,000 policy updates. Table 5 reports the total offline training time together with the latency of a single policy call.

The offline training time stays within 105.36–107.24 s across all tested noise levels. The controller overhead is also small: a single policy call takes only 0.0857–0.0928 ms on average, and the P99 latency remains below 0.17 ms in all settings. In the current simulator, each run performs 96 scheduling decisions, so the total inference overhead is negligible relative to the end-to-end execution time. Since the current evaluation is based on offline simulation rather than a live Megatron-LM integration path, we do not report additional CPU overhead relative to native Megatron-LM.

6.5. Ablation study

We next disentangle the effects of RL scheduling and computation migration. Table 6 reports the ablation results under 10% runtime noise.

RL scheduling alone reduces the completion time from 342.2315 ms to 321.2232 ms and decreases the bubble ratio from 25.15% to 20.26%, showing that dynamic scheduling removes a noticeable portion of idle time. Computation migration further reduces the completion time to 258.8029 and the bubble ratio to 1.02%. Combining the two reduces the completion time to 242.7314 ms and the bubble ratio to 0.79%, indicating that the two components are complementary.

6.6. Robustness to runtime noise and residual bubble rate

To evaluate robustness, we inject 0%–30% runtime noise and report completion time, bubble time, and residual bubble ratio in Table 7. This experiment also directly answers how much bubble remains after applying CONDUCTOR.

Static ZB-2p consistently leaves a large residual bubble ratio, ranging from 25.15% to 27.17% across the tested noise levels. RL scheduling alone reduces this ratio to 20.01%–24.01%, while computation migration reduces it to 0.43%–1.59%. Our full method further reduces the residual bubble ratio to 0.39%–1.24%. The same trend is reflected in completion time: compared with the 339.5667 ms–343.3823 ms range of ZB-2p, our method remains within 231.9042 ms–250.9180 ms. Overall, these results show that CONDUCTOR remains stable under injected runtime perturbations and keeps the residual bubble close to zero even when execution time varies substantially.

6.7. Heavy-tailed noise and network microburst stress test

To better approximate multi-tenant cloud environments, we include an offline simulator stress test with heavy-tailed runtime delays and concurrent offload contention. In this stress test, per-operation compute perturbations follow the Pareto model described in Section 6.1, and communication latency is inflated when simultaneous migrations compete for the same network path. We also include a heterogeneous-capacity stress case to evaluate the limits of the symmetric-pairing rule under mixed device speeds. These results are simulator stress evidence and should not be interpreted as held-out Megatron-LM trace calibration (see Table 8).

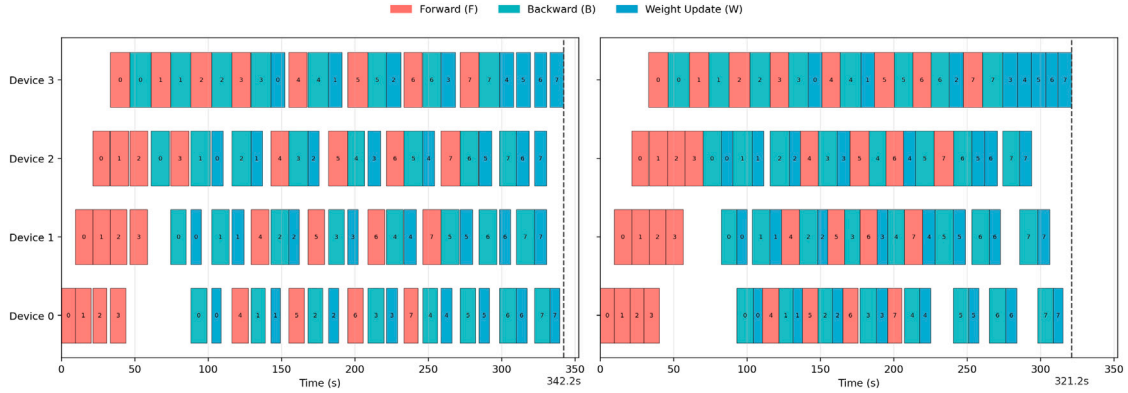
Table 3Simulator scalability stress with ZB-2p and migration. Completion time is shown as mean $\pm$ 95% CI.

Devices	Micro-batches	n	Algorithm	Completion time (ms)	Bubble ratio
4	8	3	ZB-2p	342.4980 $\pm$ 4.8804	25.23% $\pm$ 0.87 pp
4	8	3	ZB-2p+Migration	257.6673 $\pm$ 1.7586	0.62% $\pm$ 0.33 pp
8	16	3	ZB-2p	1304.5844 $\pm$ 2.5377	54.21% $\pm$ 0.17 pp
8	16	3	ZB-2p+Migration	615.2596 $\pm$ 2.0557	2.90% $\pm$ 0.31 pp
16	16	3	ZB-2p	2982.4397 $\pm$ 16.5539	74.24% $\pm$ 0.16 pp
16	16	3	ZB-2p+Migration	879.3517 $\pm$ 3.6739	12.62% $\pm$ 0.41 pp
32	16	3	ZB-2p	7925.7597 $\pm$ 19.3080	85.95% $\pm$ 0.01 pp
32	16	3	ZB-2p+Migration	1444.3586 $\pm$ 23.4411	22.87% $\pm$ 1.15 pp
64	16	1	ZB-2p	24139.3288 $\pm$ 0.0000	92.53% $\pm$ 0.00 pp
64	16	1	ZB-2p+Migration	2533.6740 $\pm$ 0.0000	28.85% $\pm$ 0.00 pp

Table 4

Megatron-LM throughput and memory for the computation-balancing module.

	Model (GPT-3)	1.5B	6.2B	14.6B
Configuration	#GPU	8	8	16
	#Microbatch	24	24	48
Samples per GPU per second	ZB-2p/Ours	14.5/15.1	4.32/4.51	1.81/1.90
	ZB-1p/Ours	12.9/13.4	3.88/4.02	1.61/1.66
	1F1B-I/Ours	13.1/13.6	4.01/4.16	1.54/1.62
	1F1B/Ours	11.8/13.0	3.50/3.78	1.40/1.57
Memory (GB)	ZB-2p/Ours	59/59	70/70	51/51
	ZB-1p/Ours	32/32	42/42	33/33
	1F1B-I/Ours	40/40	48/48	39/39
	1F1B/Ours	30/30	39/39	32/32

**Fig. 4.** Pure ZBPP ordering (no automation) vs. RL dynamic scheduling.**Table 5**

RL training time and policy inference latency under different noise levels. Best results are shown in bold.

Noise	Wall time (s)	Avg call (ms)	P99 call (ms)
0%	106.04	0.0916	0.1495
10%	106.83	0.0904	0.1529
20%	105.36	0.0857	0.1612
30%	107.24	0.0928	0.1381

Table 6

Ablation study under 10% runtime noise.

Algorithm	Completion time (ms)	Time improve vs. ZB-2p	Throughput improve vs. ZB-2p	Bubble ratio
ZB-2p	342.2315	0.00%	0.00%	25.15%
ZB-2p+RL	321.2232	6.14%	6.54%	20.26%
ZB-2p+Migration	258.8029	24.38%	32.24%	1.02%
Ours	242.7314	29.07%	40.99%	0.79%

Table 7

Robustness under runtime noise and residual bubble quantification.

Noise	Algorithm	Completion time (ms)	Bubble time (ms)	Bubble ratio
0%	ZB-2p	341.5000	86.7000	25.39%
0%	ZB-2p+RL	335.3000	80.5000	24.01%
0%	ZB-2p+Migration	255.9000	1.1000	0.43%
0%	Ours	250.9180	0.9786	0.39%
10%	ZB-2p	342.2315	86.0720	25.15%
10%	ZB-2p+RL	321.2232	65.0638	20.26%
10%	ZB-2p+Migration	258.8029	2.6435	1.02%
10%	Ours	242.7314	1.9176	0.79%
20%	ZB-2p	343.3823	93.2859	27.17%
20%	ZB-2p+RL	312.6454	62.5489	20.01%
20%	ZB-2p+Migration	254.1311	4.0347	1.59%
20%	Ours	231.9042	2.5973	1.12%
30%	ZB-2p	339.5667	88.9587	26.20%
30%	ZB-2p+RL	320.7661	70.1581	21.87%
30%	ZB-2p+Migration	254.4548	3.8468	1.51%
30%	Ours	239.8845	2.9746	1.24%

Table 8

Heavy-tailed noise, network microburst, and heterogeneous-capacity simulator stress tests. Completion time and bubble ratio are reported as mean $\pm$ 95% CI.

Stress mode	Algorithm	n	Completion time (ms)	Bubble ratio
Heavy-tail + microburst	1F1B	5	443.8579 $\pm$ 14.2407	41.02% $\pm$ 0.82 pp
Heavy-tail + microburst	ZB-2p	5	358.8103 $\pm$ 18.4667	26.96% $\pm$ 2.01 pp
Heavy-tail + microburst	ZB-2p+Migration	5	266.5548 $\pm$ 8.7966	1.79% $\pm$ 1.07 pp
Heterogeneous capacity	1F1B	5	422.8595 $\pm$ 3.2820	40.92% $\pm$ 0.30 pp
Heterogeneous capacity	ZB-2p	5	365.8911 $\pm$ 3.3611	31.72% $\pm$ 0.27 pp
Heterogeneous capacity	ZB-2p+Migration	5	255.1455 $\pm$ 3.0360	2.07% $\pm$ 0.58 pp

Table 9

Speedup (%) with hidden dimension (h) and sequence length (L) at $\alpha = 0.1$.

Sequence length (L)	$h = 2048$	$h = 4096$	$h = 5120$
1024 (1k)	5.1%	7.2%	8.5%
2048 (2k)	6.2%	8.5%	9.4%
4096 (4k)	7.0%	9.2%	9.8%
8192 (8k)	7.3%	9.6%	10.2%
16,384 (16k)	6.8%	8.9%	9.5%
32,768 (32k)	5.5%	7.8%	8.7%

Under heavy-tail + microburst stress, ZB-2p+Migration reduces completion time from 358.8103 $\pm$ 18.4667 ms to 266.5548 $\pm$ 8.7966 ms and reduces residual bubble ratio from 26.96% $\pm$ 2.01 percentage points to 1.79% $\pm$ 1.07 percentage points. Under heterogeneous-capacity stress, it reduces completion time from 365.8911 $\pm$ 3.3611 ms to 255.1455 $\pm$ 3.0360 ms and residual bubble ratio from 31.72% $\pm$ 0.27 percentage points to 2.07% $\pm$ 0.58 percentage points.

6.8. Micro-analysis of computation balancing

To further examine when migration is most effective, we vary the hidden dimension and sequence length and use MLP transfer as a representative decomposable computation unit. We adopt a single-layer Transformer as a controlled case study to validate the communication and overlap mechanism. Table 9 reports the resulting speedup when the reducible fraction is fixed at $\alpha = 0.1$.

The benefit first increases with sequence length and hidden size, reaches its maximum around the 8k sequence-length regime, and then decreases slightly when communication overhead becomes more prominent. This trend is consistent with the design of computation balancing: larger decomposable operators provide more room for beneficial migration, but the gain is ultimately bounded by transfer cost.

7. Limitations and future work

The current study uses a hybrid evaluation: the RL scheduler is tested in a hardware-profile-anchored simulator, whereas computation migration is measured in Megatron-LM on up to 16 GPUs. This division reflects their control timescales: RL is used for global scheduling, while deterministic equations provide the low-overhead response required for intra-slot migration. The results therefore validate the two components separately rather than a fully integrated runtime. The simulator does not yet have held-out trace calibration, and the repeated-seed analysis covers only representative simulator settings. Moreover, symmetric pairing is best suited to modest device heterogeneity, while highly heterogeneous clusters would require capacity-aware matching. The present results should consequently be interpreted as evidence of mechanism feasibility rather than industrial-scale deployment.

Future work will integrate both components into a single Megatron-LM execution path and evaluate it on larger clusters. We will also quantify simulator-to-hardware agreement using matched traces, extend statistical evaluation and aligned adaptive-baseline comparisons, and profile optimizer synchronization, temporary buffers, collective operations, and network contention. These additional costs can be

incorporated into Ω_{unmasked} to adjust the admissible migration fraction. Thus, CONDUCTOR aims to minimize residual bubbles rather than guarantee zero-bubble execution under arbitrary system conditions.

8. Conclusion

In this paper, we presented CONDUCTOR, a dynamic multi-granularity control framework for mitigating stragglers in pipeline-parallel training. The framework separates coarse-grained schedule optimization from fine-grained computation balancing, allowing the two sources of pipeline inefficiency to be addressed at their corresponding control timescales.

The simulator experiments show that the RL scheduler can improve global operation ordering under the tested runtime perturbations, while the Megatron-LM experiments show that the computation-migration module improves throughput by 5%–14% over the evaluated static schedules without increasing peak memory. These results support the effectiveness of the two component mechanisms, although the present evidence does not yet constitute a fully integrated RL-plus-migration deployment.

Overall, the results indicate that combining learned global scheduling with deterministic local migration is a promising direction for adaptive pipeline-parallel training. The conclusions are limited to the simulator conditions and module-level hardware configurations evaluated in this study.

CRedit authorship contribution statement

Xingbo Dong: Writing – review & editing, Writing – original draft, Supervision, Methodology, Funding acquisition, Formal analysis. **Ziyuan Liu:** Writing – review & editing, Writing – original draft, Methodology. **Yuezhe Yang:** Writing – review & editing, Writing – original draft, Methodology, Conceptualization. **Yen-Lung Lai:** Writing – review & editing, Supervision. **Zhe Jin:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- [1] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q.V. Le, Y. Wu, et al., Gpipe: Efficient training of giant neural networks using pipeline parallelism, *Adv. Neural Inf. Process. Syst.* 32 (2019).
- [2] S. Fan, Y. Rong, C. Meng, Z. Cao, S. Wang, Z. Zheng, C. Wu, G. Long, J. Yang, L. Xia, et al., DAPPLE: A pipelined data parallel approach for training large models, in: *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2021, pp. 431–445.
- [3] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N.R. Devanur, G.R. Ganger, P.B. Gibbons, M. Zaharia, PipeDream: Generalized pipeline parallelism for DNN training, in: *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019, pp. 1–15.
- [4] S. Ao, W. Zhao, X. Han, C. Yang, X. Zhang, Z. Liu, C. Shi, M. Sun, Seq1F1B: Efficient sequence-level pipeline parallelism for large language model training, in: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2025, pp. 8998–9008.
- [5] J. Lin, Z. Jiang, Z. Song, S. Zhao, M. Yu, Z. Wang, C. Wang, Z. Shi, X. Shi, W. Jia, et al., Understanding stragglers in large model training using what-if analysis, in: *19th USENIX Symposium on Operating Systems Design and Implementation, OSDI 25*, 2025, pp. 483–498.

- [6] S. Li, H. Ma, Z. Wang, X. Zhang, A survey of machine learning approaches to solving NP-hard problems, *Int. J. Comput. Sci. Math.* 22 (1) (2025) 1–31.
- [7] D. Narayanan, A. Phanishayee, K. Shi, X. Chen, M. Zaharia, Memory-efficient pipeline-parallel dnn training, in: *International Conference on Machine Learning*, PMLR, 2021, pp. 7937–7947.
- [8] P. Qi, X. Wan, G. Huang, M. Lin, Zero bubble (almost) pipeline parallelism, in: *The Twelfth International Conference on Learning Representations*, 2024.
- [9] H. Mishmast Nehi, H.A. Ashayerinasab, M. Allahdadi, Solving methods for interval linear programming problem: a review and an improved method, *Oper. Res.* 20 (3) (2020) 1205–1229.
- [10] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, et al., Deepseek-v3 technical report, 2024, arXiv preprint arXiv:2412.19437.
- [11] M. Wahib, M.A. Soyuturk, D. Unat, Balanced and elastic end-to-end training of dynamic LLMs, in: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 1351–1367.
- [12] S. Li, T. Hoefler, Chimera: efficiently training large-scale neural networks with bidirectional pipelines, in: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–14.
- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [14] J.H. Park, G. Yun, M.Y. Chang, N.T. Nguyen, S. Lee, J. Choi, S.H. Noh, Y.r. Choi, {HetPipe}: Enabling large {DNN} training on (whimpy) heterogeneous {GPU} clusters through integration of pipelined model parallelism and data parallelism, in: *2020 USENIX Annual Technical Conference, USENIX ATC 20*, 2020, pp. 307–321.
- [15] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, et al., Efficient large-scale language model training on gpu clusters using megatron-lm, in: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–15.
- [16] Q. Wang, J. Yang, X. Tan, M. Sha, Z. Ling, H. He, X. Jiang, iPipe: An efficient adaptive pipeline parallel DNN training system, in: *Proceedings of the 2024 International Symposium on AI and Cybersecurity*, 2024, pp. 50–56.
- [17] A. Dutta, N. Chaki, R.K. De, Timeprest: Time and memory efficient pipeline parallel dnn training with removed staleness, *Future Gener. Comput. Syst.* 178 (2026) 108260.
- [18] T. Wu, L. Cao, H. Lu, X. Jiang, Y. Yu, S. Yang, G. Yang, J. Wang, L. Qu, L. Zhang, et al., Adaptra: Straggler-resilient hybrid-parallel training with pipeline adaptation, 2025, arXiv preprint arXiv:2504.19232.
- [19] X. Zhang, X. Wang, X. Wang, A reinforcement learning-driven task scheduling algorithm for multi-tenant distributed systems, in: *2025 5th International Conference on Intelligent Communications and Computing, ICICC, IEEE*, 2025, pp. 197–201.
- [20] H. Mao, M. Schwarzkopf, S.B. Venkatakrishnan, Z. Meng, M. Alizadeh, Learning scheduling algorithms for data processing clusters, in: *Proceedings of the ACM Special Interest Group on Data Communication*, 2019, pp. 270–288.
- [21] J. Santos, M. Zaccarini, F. Poltronieri, M. Tortonesi, C. Sleianelli, N. Di Cicco, F. De Turck, Efficient microservice deployment in kubernetes multi-clusters through reinforcement learning, in: *NOMS 2024-2024 IEEE Network Operations and Management Symposium, IEEE*, 2024, pp. 1–9.
- [22] S. Qi, S. Zhang, Q. Wang, J. Zhang, X. Wang, Distributed scalable multi-agent reinforcement learning with intrinsic-episodic dual exploration, *Future Gener. Comput. Syst.* 175 (2026) 108040.
- [23] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, 2017, arXiv preprint arXiv:1707.06347.
- [24] Z. Li, Y. Liu, W. Zhang, T. Yuan, B. Chen, C. Song, SlimPipe: Memory-thrifty and efficient pipeline parallelism for long-context LLM training, in: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 1409–1428.
- [25] T. Brown, B. Mann, N. Ryder, M. Subbiah, J.D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, *Adv. Neural Inf. Process. Syst.* 33 (2020) 1877–1901.